Predicting Loan Default on Lending Club Data

Application-time scoring with out-of-time validation

[Student 1], [Student 2]

School of Technology, National Economics University
Statistical Learning (DS66A / DS66B) | Supervisor: [Name]

October 30, 2026



Outline

1. Problem statement
2. Objectives and contributions
3. Data and target definition
4. Out-of-time split
5. Leakage control and cleaning
6. Feature selection and engineering
7. Models, tuning and metrics
8. Validation and test results
9. Experiment 1: without `sub_grade`
10. Experiment 2: compact scorecard
11. Conclusion, limitations, future work

Problem statement

Lending Club (US peer-to-peer lending) published its loan records. Each loan ends as Charged Off (default, label 1) or Fully Paid (label 0).

Question. Using only information known at application time, how well can we *rank* applicants by default risk, and does a model beat the platform's own `sub_grade` (A1 to G5)?

Why it is hard

- **Leakage**: many of the 151 columns are created after the loan starts
- **Time**: default rate drifts (13.7% $\rightarrow$ 14.8%)
- **Censoring**: recent loans have no final outcome yet
- **Incumbent**: `sub_grade` is already a risk model

Task

Input: borrower and credit-bureau data at application
Output: score $p \in [0,1]$, the estimated probability of default, used to rank loans
Label: Charged Off $= 1$, Fully Paid $= 0$

Key rule

No data created after approval.

Objectives and contributions

Objective: estimate honestly how much application-time information adds to `sub_grade` for ranking default risk.

Contributions

- Leakage-controlled protocol: application-time whitelist, evidence from single-column AUC
- Out-of-time 70/15/15 split on matured 36-month loans, with a fixed split manifest
- Pipeline fitted on train only (median impute, scaling, one-hot)
- Comparison of baseline, Logistic L2, Decision Tree, Random Forest, HistGB with ROC-AUC, Gini, KS and paired bootstrap
- Two follow-ups: removing `sub_grade`, and a compact 12-field scorecard
- Negative results reported as they are

Data dictionary: which columns are known at application?

Kaggle file: **2,260,701** loans, **151** columns (2007–2018). Every column is tagged with a hypothesis.

Group	Columns
Credit history (1)	38
Credit history (2)	31
Loan information	21
Joint application	16
Post-loan payments	15
Hardship plan	15
Borrower information	8
Debt settlement	7

Timing tags

Yes: 103 known at application
No: 41 after the loan, or identifiers
Uncertain: 7 timing unclear

- Dictionary matches the raw header: 151 / 151, none missing, extra or duplicated
- The tags are hypotheses; they are tested with data later (single-column AUC)

Target and population

Label from `loan_status`: Fully Paid $\rightarrow 0$, Charged Off $\rightarrow 1$. A loan is final only if no principal is outstanding (checked with `out_prncp`).

Filters (each verified by code)

- labelled
- issued from 2012 (pre-2012 is 2.96% of labelled loans; 44 numeric columns start in 2012)
- matured: issue month + term $\leq$ 2018-12 (99.6% of matured loans are labelled vs 42.4%)
- 36-month term (all three sets share one term)

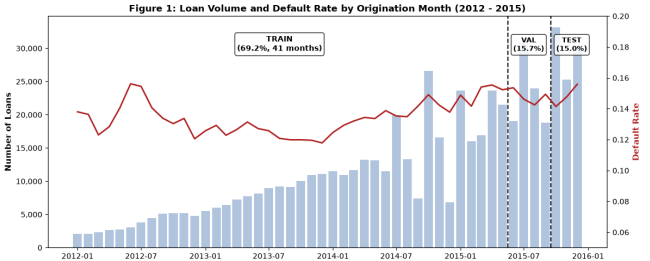
Step	Rows	Kept
Raw data	2,260,701	100%
Labelled	1,345,310	59.5%
From 2012	1,305,524	57.7%
Matured	633,767	28.0%
36-month	**589,488**	**26.1%**

Among labelled loans: 80.0% Fully Paid, 20.0% Charged Off (14.0% after filtering).

Out-of-time split: train on the past, test on the future

Set	Rows	Period	Default
Train	408,188	2012-01 to 2015-05	13.7%
Val	92,631	2015-06 to 09	14.7%
Test	88,669	2015-10 to 12	14.8%

- Sort by issue date; cut at the dates found at 70% and 85% of rows
- Whole months stay in one set; assertions check train $<$ val $<$ test
- Default rate drifts up $+1.1$ points
- Manifest stores an MD5 of train ids



Leakage control and cleaning

Six cleaning rules (row by row)

- `annual_inc` ≤ 0, `loan_amnt` $\leq 0 \rightarrow$ missing
- `dti` outside 0–100 $\rightarrow$ missing
- `revol_util` capped at 102.4 (train 99.9th pct)
- FICO low $>$ high $\rightarrow$ both missing
- first credit line after issue date $\rightarrow$ missing

Anti-leakage rules

1. Split first, transform later
2. Median, mean, std, caps, categories: **train only**
3. Pipeline `fit` on train; val and test only `transform`
4. Same cleaning function on all three sets

Evidence: after scaling, train means are ≈ 0 but validation means reach 0.162, because validation uses train statistics.

Feature selection: 151 → 57 columns

Rule (in order)	Columns
Raw columns	151
Label and helper	−2
Identifier	−2
Empty in train	−32
After loan / unclear	−42
Text > 100 values	−4
Near constant	−7
Missing > 30%	−5
Candidates	**57**

Leakage check: single-column ROC-AUC on train

Best application-time column: `sub_grade`, **0.6565**.

Suspect column	AUC
`last_fico_range_low`	0.9074
`recoveries`	0.9054
`collection_recovery_fee`	0.8677
`total_rec_prncp`	0.8280
`total_pymnt`	0.7554

10 of the 42 suspect columns beat the best legitimate column: the whitelist is justified.

Feature engineering and pipeline: 57 → 153 columns

New features: `fico_avg`, `loan_to_income` (cap 0.5), `log_annual_inc`, `log_revol_bal`, `emp_length_years` (and `credit_history_years`, later dropped for $r = 0.92$).

Dropped: replaced columns, `grade` and `int_rate` (duplicate `sub_grade`), and 5 columns with $|r| \geq 0.90$.

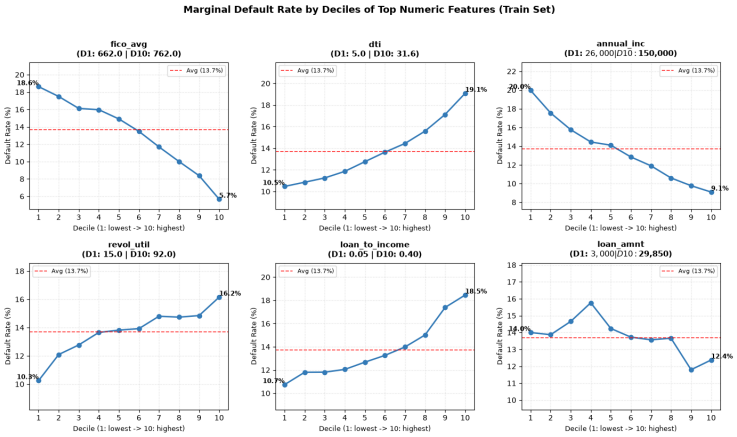
Result: 45 numeric + 5 categorical = 50 inputs → **153** columns after one-hot (108 dummies: `addr_state` 50, `sub_grade` 35, `purpose` 14, `home_ownership` 6, `verification` 3).

Why median and scaling?

- 28 of 31 columns with missing values are right-skewed; mean is pulled up (e.g. `avg_cur_bal`: mean 12,503 vs median 6,228)
- Scales differ by $> 10^5$ (0.34 to 127,162), which matters for the L2 penalty

`Pipeline: clean → features → ColumnTransformer(median+StandardScaler | OneHot)`

Default rate by decile (train): signal and non-linearity



- FICO, income: risk falls steadily
- DTI, utilisation, loan-to-income: risk rises
- `loan_amnt` is not monotone (peak 15.7%), which a linear term cannot capture
- `sub_grade`: default rate rises from 2.7% (A1) to 36.4% (G5)

Models, tuning and metrics

Five models, 153 inputs (baseline uses `sub_grade` only). Tuned by `GridSearchCV` with `TimeSeriesSplit(3)` inside train.

Model	Selected	CV AUC
Logistic L2	$C = 1$	0.6855
Decision Tree	depth 7	0.6355
Random Forest	150 trees, depth 10	0.6741
HistGB	150 iter, depth 10, lr 0.05	0.6852

Metrics (ranking, no threshold)

ROC-AUC: chance a defaulter scores above a repaid loan

Gini $= 2\,\mathrm{AUC} - 1$

KS $= \max(\mathrm{TPR} - \mathrm{FPR})$

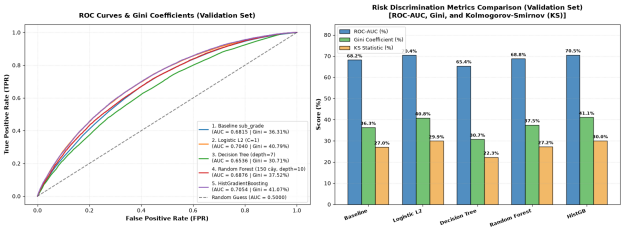
- Overfit check: AUC train − AUC val
- Paired bootstrap, $B = 1{,}000$, 95% CI of ΔAUC
- Test used once

Validation results (92,631 loans)

Model	AUC	Gap	Gini	KS
Baseline	0.6815	−0.025	36.3%	27.0%
Logistic	0.7040	−0.017	40.8%	29.9%
Tree	0.6536	+0.001	30.7%	22.3%
Random Forest	0.6876	+0.018	37.5%	27.2%
HistGB	0.7054	+0.004	41.1%	30.0%

Bootstrap ΔAUC vs baseline

- Logistic +0.0225 [+0.0196, +0.0253]
- HistGB +0.0238 [+0.0206, +0.0270]
- RF +0.0061 [+0.0024, +0.0102]
- Tree −0.0280 [−0.0329, −0.0233]



Out-of-time test and decision

Test: 88,669 loans (2015-10 to 12), evaluated once.

Model	AUC	Gini	KS	ΔAUC
Baseline `sub_grade`	0.6818	36.4%	26.5%	—
Logistic L2	**0.7039**	40.8%	29.9%	+0.0221
HistGB	0.7055	41.1%	30.0%	+0.0237

HistGB − Logistic on validation: 95% CI [−0.0002, +0.0031] contains 0, so no real difference.

Decision: Logistic L2

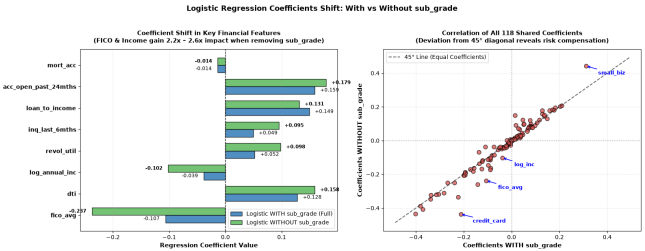
- beats baseline by +0.022 AUC
- same level as HistGB
- fast, no overfitting
- each coefficient is explainable

Choosing Logistic is a modelling judgement, not a statistical result.

Experiment 1: remove `sub_grade` (153 → 118 features)

Model	Val AUC	Test AUC	Test KS
Baseline	0.6815	0.6818	26.5%
Logistic, with	0.7040	0.7039	29.9%
Logistic, without	0.6890	0.6879	27.3%

- Without the grade, still above baseline: +0.0075 [+0.0033, +0.0117]
- `sub_grade` adds +0.0150 [+0.0133, +0.0167]
- Coefficients grow when the grade is removed: `fico_avg` 2.2×, `log_annual_inc` 2.6×, `revol_util` 1.9×



The grade absorbs signal from these variables (association, not causation).

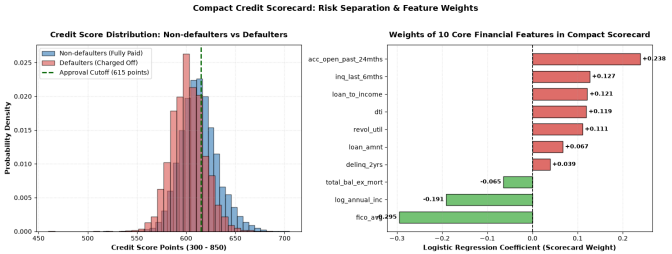
Experiment 2: compact 12-field scorecard

Model	Cols	Test AUC	Gini
Full Logistic	153	0.7039	40.8%
Compact	30	0.6730	34.6%

Compact keeps 84.8% of the full Gini with 80% fewer columns but is below the baseline (0.6818).

Score: Offset $-$ Factor $\cdot \ln(\text{odds})$, Factor $= \text{PDO}/\ln 2$; PDO 20, 600 points at odds 1/5 (conventions we chose).

Band	Share	Default
< 590	11.4%	30.2%
590–614	47.6%	17.3%
615–639	33.5%	8.7%
≥ 640	7.5%	3.3%



Test set; cut-offs set on validation.

Conclusion

1. **The incumbent is beaten, modestly.** Test AUC: baseline 0.6818, Logistic 0.7039, HistGB 0.7055; intervals on validation exclude zero.
2. **More complex is not clearly better.** HistGB and Logistic tie; the forest gains less; a depth-7 tree loses to the baseline.
3. **The grade absorbs signal.** Without `sub_grade`: test AUC 0.6879 (+0.0061 over baseline); FICO and income coefficients about double.
4. **A compact scorecard is cheap but weaker**, yet separates risk bands from 30.2% to 3.3% observed default.
5. Every number is printed by code on real data, and disappointing results are reported as such.

Limitations and future work

Limitations

- 36-month loans of 2012–2015; three-month test window
- Random Forest and HistGB grids hit their edges
- Bootstrap CI covers validation sampling only
- Val AUC $>$ train AUC for the baseline and Logistic: observed, not explained
- No calibration, thresholds or cost analysis
- Scorecard constants (600, PDO 20) are conventions

Future work

- Wider or randomised hyperparameter search
- Rolling-origin evaluation and dataset-shift tests [1]
- Missing-indicator treatment of `mths_since_...` columns
- Calibration and cost-sensitive thresholds
- 60-month loans via survival analysis
- Fairness and stability checks

References I

H. Qian, B. Wang, P. Ma, L. Peng, S. Gao, and Y. Song, "Managing dataset shift by adversarial validation for credit scoring," arXiv preprint arXiv:2112.10078, 2021.

C. Serrano-Cinca, B. Gutiérrez-Nieto, and L. López-Palacios, "Determinants of default in P2P lending," *PLoS ONE*, vol. 10, no. 10, p. e0139427, 2015.

M. Malekipirbazari and V. Aksakalli, "Risk assessment in social lending via random forests," *Expert Systems with Applications*, vol. 42, pp. 4621–4631, 2015.

N. Siddiqi, *Intelligent Credit Scoring: Building and Implementing Better Credit Risk Scorecards*, 2nd ed., ser. Wiley and SAS Business Series. Hoboken, NJ: John Wiley & Sons, 2017.

N. George, "Lending club loan data (accepted and rejected loans, 2007–2018)," Kaggle dataset (user `wordsforthewise`), https://www.kaggle.com/datasets/wordsforthewise/lending-club, accessed 2026.

D. J. Hand and W. E. Henley, "Statistical classification methods in consumer credit scoring: a review," *Journal of the Royal Statistical Society: Series A*, vol. 160, no. 3, pp. 523–541, 1997.

References II

F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

J. H. Friedman, "Greedy function approximation: a gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: a highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 3146–3154.

B. Efron, "Bootstrap methods: another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.

T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.

References III


F. J. Massey, "The Kolmogorov–Smirnov test for goodness of fit," *Journal of the American Statistical Association*, vol. 46, no. 253, pp. 68–78, 1951.

Thank you

Questions and discussion